

TPM FUTURE LAB · PRODUCT × AI × AWS

HomeWellness Companion

主動式健康關懷與用藥提醒

產品設計、AI 工作流、AWS 架構與 MVP Demo
報告

主動

可信

可追溯

DEMO REPORT · INTERVIEW EDITION

目錄

1. Executive Summary
2. 王伯伯的故事、使用者與核心洞察
3. MVP 範圍與目前完成度
4. 系統架構與技術取捨
5. AI Agent 工作流
6. Live Demo 腳本 (建議 4 分鐘)
7. 安全、信任與隱私界線
8. 工程證據
10. 下一階段 Roadmap
11. 常見 Q&A
13. 資料來源與盤點限制

1. Executive Summary

HomeWellness Companion 是一個主動式居家健康關懷與用藥提醒 MVP。它把用藥計畫、時間與 IoT 情境、語音 Agent 概念及規則引擎串成一條可展示流程：

家屬建立並確認藥單 → 系統在晚餐情境主動詢問 → 記錄服藥結果 → 未回應時逐級提醒與通知家屬。

核心價值不是再做一個固定時間鬧鐘，而是「在適合的生活情境發起對話，確認結果，並保留可信程度」。系統刻意把對話生成與醫療規則分開：LLM 負責自然互動，風險分級、提醒升級及資料有效性由可測試的程式規則決定。

目前最穩定的展示範圍是 Memory backend 下的藥單模擬 OCR、人工校對與啟用、IoT / 時間觸發、服藥狀態與逾時通知。完整 Gemini 對話及真實 DynamoDB 尚未在目前環境驗證，報告中應標示為「已有介面 / 實作，待憑證實證」，不能宣稱已完成雲端部署。

題目要求對照

題目要求	專案現況	判定	簡報處理方式
IoT 數據觸發主動對話	已有晚餐區事件與 18:00–19:00 時間窗	已完成	Live Demo 直接展示
長者友善的對話 Vibe	System Prompt 已定義溫暖、簡短、不責備與一次一問	已完成	放一頁原則與對話範例
User Journey / User Story / PRD Snippet	PRD.md 已涵蓋 US-01 ~ US-05	已完成	濃縮成一頁旅程與五個 Stories
LangChain Agent 工作流	已用 LangChain create_agent 串接 Gemini 與 Tools	已完成、待 Live 驗證	展示實際程式與 Tool Calling
Memory：歷史健康與用藥	Repository 可保存健康 / 用藥資料；聊天只在 session，Agent 尚無跨會話摘要	部分完成	誠實標示 MVP Memory 與下一步
頭暈時讀取血壓 Tool	get_current_vitals 會讀血壓、心率、來源與量測時間	已完成、待 Live 驗證	作為主要對話 Demo
AWS IoT Core / Lambda / DynamoDB 架構	DynamoDB adapter 與建表腳本已有；IoT Core、Lambda 尚未部署	設計完成、部署未完成	架構圖區分 PoC 與 Production Target

題目要求	專案現況	判定	簡報處理方式
可運作的 Streamlit PoC	app.py 可運行，規則流程不需外部服務	已完成	3–5 分鐘 Live Demo
簡報 / PDF	10 頁網頁簡報、投影片 PDF 與本報告 PDF 已產出	已完成 (PPTX 為選用備份)	固定 16:9 Web Deck，PDF 已逐頁檢查
GitHub Repo / Workflow	Git repository 已存在；交付版本尚待整理	部分完成	合併分支、補證據與 README 後交付

題目背景也提到心率、睡眠品質與環境溫度。現有 PoC 已涵蓋血壓與心率，但尚未實作睡眠品質與環境溫度；它們不影響本次「主動關懷與用藥提醒」主線，可在架構圖中列為裝置輸入與下一階段能力。

2. 王伯伯的故事、使用者與核心洞察

開場情境 (MVP 虛構人物)

晚間 7:30，72 歲、獨居的王伯伯進入晚餐後用藥時間窗，但系統仍沒有收到服藥確認；平常協助他確認藥單的女兒此刻正在開會。當 Agent 主動詢問時，王伯伯回答：「我剛吃完飯，可是今天有點頭暈。」

這一刻的產品問題不是「如何再響一次鬧鐘」，而是：Agent 應該先關心、讀取數據、記錄結果，還是通知家屬？這個情境會貫穿 User Journey、Live Demo、安全設計與技術架構。

核心 User Story

身為需要長期服藥的長者，我希望有人在適當時間溫和確認我的狀態；當我不舒服或忘記服藥時提供協助，讓我安心生活，也不讓家人過度擔心。

目標使用者

- 主要使用者：獨居或白天獨處、需要規律服藥的長者。
- 次要使用者：無法時刻陪伴、但希望掌握異常與漏服情況的家屬。

現有問題

- 固定鬧鐘不知道使用者是否正在用餐，也無法確認提醒後的結果。
- 長者可能忘記、延後或只口頭表示已完成，家屬缺少分級且可信的資訊。
- LLM 若直接判定醫療風險或自行補出用藥內容，會產生高風險錯誤。

產品洞察

IoT 訊號不應直接等同「已吃飯」或「已吞服」，而應作為發起詢問或提高紀錄可信度的依據。系統因此保留 `self_reported` 與 `sensor_supported` 等不同狀態，而不是把所有結果簡化成單一完成 / 未完成。

對話 Vibe

- 溫暖但不過度擬人化，不假裝成家人或醫療專業人員。
- 一次只問一件事，句子短、選項少，降低長者理解與回覆負擔。
- 不責備、不說教；漏服時使用關懷式確認，不使用恐嚇語言。
- 重要資訊重述確認，並清楚說明資料來源、量測時間與不確定性。
- 危險警訊使用直接、可回答「有 / 沒有」的問法，必要時建議聯絡真人或緊急服務。

範例：

「王伯伯，您吃完晚餐了嗎？吃完後請依已確認藥單服用脈優錠 5mg 1 顆。」 「您剛才提到頭暈。現在有胸痛、呼吸困難，或意識不清楚嗎？」

Supporting User Stories

- **US-01 建立可信藥單**：家屬可以校對 OCR 候選並逐筆確認，避免錯誤內容直接進入提醒。
- **US-02 主動確認用餐**：當時間與 IoT 情境適合時，Agent 主動詢問是否已用餐，但不自行判定。
- **US-03 提醒並記錄服藥**：長者可以口頭回報，系統保留回報來源與可信程度。
- **US-04 身體不適時讀取健康資料**：長者表示頭暈時，Agent 先確認危險警訊，再透過 Tool 讀取血壓與心率。
- **US-05 未完成時逐級通知**：未回應或持續漏服時，系統依固定規則提醒並通知家屬。

PRD Snippet

- **目標**：驗證情境式主動關懷是否比固定鬧鐘更能促成回應與完成紀錄。
- **觸發**：晚餐時間窗或餐桌區感測事件；同一事件不可重複觸發相同訊息。
- **資料條件**：只允許使用已確認、已啟用且在有效期間內的用藥計畫。
- **結果狀態**：區分排程、到期、已提醒、口頭回報、感測支持及漏服。
- **升級**：依 20 / 60 / 120 分鐘門檻進行提醒、一般家屬通知與高優先通知。
- **非目標**：不診斷、不調藥、不證明已吞服、不以單一感測數值直接判定生活行為。

3. MVP 範圍與目前完成度

能力	目前狀態	可展示證據	報告說法
藥單拍攝 / 上傳入口	已完成	Streamlit 相機、上傳與示範藥單入口	UI 可接收圖片，但不送往外部 OCR
模擬 OCR 與可信來源圖片	已完成	固定辨識候選、信心值、精確藥名圖片來源	情境模擬，不宣稱真實 OCR
家屬逐筆校對、增刪與啟用	已完成	必填驗證、固定時間驗證、逐筆確認	未確認資料不能進入提醒流程
用藥計畫稽核紀錄	已完成	建立、修改、刪除、核對與啟用事件	可追溯操作者、時間與前後內容
IoT / 時間主動觸發	已完成	晚餐感測事件及 18:00-19:00 時間窗	同一事件 / 同一天避免重複詢問
規則式服藥升級	已完成	20、60、120 分鐘狀態與通知	規則與 LLM 分離，可獨立測試
LangChain + Gemini 對話	條件式完成	Agent、Prompt、四個 Tools 已接線	需有效 API Key 與現場模型可用性驗證
Memory backend	已完成	無外部憑證可執行規則流程	最穩定的 Demo 模式
DynamoDB adapter / 建表腳本	已實作	單表 PK/SK 設計與 Repository adapter	AWS 已部署驗證
跨 Session Agent Memory	尚未完成	對話只存在 Streamlit session	列為下一階段
正式通知服務	尚未完成	目前只在側欄模擬家屬通知	SNS / 推播屬 Production Target

4. 系統架構與技術取捨



模組責任

- `domain/`：用藥計畫、生命徵象、IoT 事件、服藥紀錄、通知及狀態列舉。
- `storage/`：Repository 抽象，以及 Memory / DynamoDB 可切換實作。
- `rules/`：生理風險、主動觸發與服藥逾時升級規則。
- `simulator/`：可重現的模擬時間、IoT 事件與藥單 OCR 情境。
- `agent/`：Gemini 模型、System Prompt、LangChain Agent 與 Tools。
- `app.py`：Streamlit UI、session state、情境按鈕及展示資訊。

關鍵取捨

1. 先用 Streamlit 驗證完整故事：開發快、操作直觀，適合 3–5 分鐘 PoC；不是 Production 前端架構。
2. Repository 抽象隔離資料層：開發模式用 Memory 保證 Demo 穩定，取得 AWS 憑證後再切 DynamoDB。
3. 規則引擎掌握安全決策：LLM 不自行制定血壓、心率或漏服升級規則。
4. 模擬 OCR 而不假裝真實能力：先驗證人工核對、資料治理與後續提醒的價值鏈。

AWS Production Target 架構

下圖是題目要求的 AWS 目標架構；虛線概念在目前 PoC 尚未部署，正式投影片應以不同顏色標示「已實作」與「規劃中」。

HomeWellness Device

心率／血壓／睡眠／環境感測器、麥克風、智慧藥盒

MQTT over TLS

AWS IoT Core — Device Shadow

IoT Rules → Lambda / EventBridge → DynamoDB

事件觸發

健康、用藥、事件、稽核

LangChain Agent Service
(ECS Fargate / App Runner)

SNS / Push → 家屬端

Tools / Rules Engine

Gemini (目前)

或 Amazon Bedrock (選配)

共通能力：Cognito／IAM 最小權限、KMS 加密、CloudWatch Logs／Metrics、
Secrets Manager、API Gateway / WebSocket

PoC 與 Production Target 邊界

區域	PoC 已實作	Production Target
裝置事件	Python 模擬餐桌與藥盒事件	真實感測器透過 MQTT 傳送
事件入口	Streamlit 按鈕與模擬時間	AWS IoT Core、IoT Rules、EventBridge
Agent 執行	本機 Streamlit process	ECS Fargate / App Runner 常駐服務
資料	Memory ; DynamoDB adapter 已寫	真實 DynamoDB、備份、TTL 與存取控制
通知	UI 側欄模擬	SNS / 行動推播 / 照護平台
觀測與安全	應用錯誤與稽核事件	CloudWatch、KMS、Secrets Manager、Cognito、IAM

5. AI Agent 工作流

1. 時間窗或晚餐區 IoT 事件觸發「現在適合詢問」。
2. 系統主動問：「王伯伯，您吃完晚餐了嗎？」而不是直接斷言已用餐。
3. Agent 只透過 `get_medication_plan` 取得已確認、啟用且仍有效的藥單。
4. 若使用者提及頭暈等不適，Agent 先確認胸痛、呼吸困難等危險警訊。
5. `get_current_vitals` 把生理數據與警訊確認結果交給規則引擎分級。
6. 使用者口頭回報服藥時，`record_dose_self_report` 記錄為 `self_reported`。
7. 藥盒開啟與重量下降可將紀錄提高為 `sensor_supported`，但不等於證明吞服。
8. 若持續未回應，規則引擎依時間門檻提醒並產生家屬通知。

Agent 已提供四個工具：

- `get_current_vitals`
- `get_medication_plan`
- `record_dose_self_report`
- `get_dose_history`

6. Live Demo 腳本 (建議 4 分鐘)

Demo 前畫面

- 使用 `DATA_BACKEND=memory`，先確認側欄顯示 `Backend: memory`。
- 若要展示完整對話，必須先驗證 Gemini Key、模型與網路。
- 若 LLM 不可用，改走純規則備援路線，清楚說明「現在展示的是可重現的規則與事件流程」。

成功主線

時間	操作	講者重點
0:00–0:20	說明畫面與模擬時間	「這是文字模擬語音的 PoC，左側是家屬與情境控制，右側是長者對話。」
0:20–1:20	執行模擬 OCR、展開校對、逐筆確認並啟用	強調 OCR 不可信任到可直接使用，必須經人工核對與稽核
1:20–2:00	點擊晚餐感測事件	強調事件只觸發詢問，不直接判定已用餐
2:00–3:00	有 LLM 時輸入「吃完了，但今天有點頭暈」	展示危險警訊追問、生命徵象 Tool 與規則分級
3:00–3:30	回報已服藥	說明狀態是口頭回報 <code>self_reported</code>
3:30–4:00	點擊藥盒開啟 + 重量下降	說明感測訊號提高可信度，但不能證明吞服

未回應分支

若要展示家屬通知，晚餐事件後不回覆，連續按「前進 30 分鐘」：

- 約 18:30：第一次提醒，狀態轉為 `reminded`。
- 約 19:00：狀態轉為 `missed`，產生 medium 通知。
- 約 20:00：仍未回應，升級 high 通知。

建議主 Demo 只選「成功服藥」或「未回應升級」其中一條；另一條放在備用畫面或 Q&A，避免超時。

7. 安全、信任與隱私界線

- 產品提供提醒與資訊協助，不診斷、不調藥、不取代醫師、藥師或緊急醫療服務。
- 風險等級由規則引擎輸出，LLM 只解釋結果與維持對話。
- 未確認、未啟用、尚未生效或已過期的用藥計畫不會進入提醒。
- IoT 訊號只作情境與可信度證據，不等同已用餐或已吞服。
- 相機 / 上傳影像目前不送往外部服務；模擬 OCR 必須清楚揭露。
- 所有藥單候選的建立、修改、刪除、核對與啟用均保留稽核事件。
- DynamoDB 寫入失敗會拋出錯誤，不會在 UI 假裝成功。

8. 工程證據

本次盤點實際執行測試：

```
69 passed, 2 warnings in 16.28s
```

- 40 個 Python 檔：26 個產品 / 腳本檔、14 個測試檔。
- 約 1,687 行產品程式與 710 行測試程式。
- 測試涵蓋 OCR 驗證與稽核、有效用藥篩選、IoT 事件、時間窗觸發、升級規則、Memory repository、Agent tools 與 Streamlit smoke test。
- 兩個 warning 分別來自第三方 Google GenAI 套件棄用提示，以及受限環境無法寫入 pytest cache；測試本身全數通過。

10. 下一階段 Roadmap

P1：技術實證

- 用最小權限 IAM 在真實 DynamoDB 完成端到端讀寫驗證，保留 Request ID / Console 證據。
- 區分短期對話歷史與長期健康事件摘要，完成跨 Session Memory；讓 Agent 能回答近期健康趨勢與過去漏服情況，而不把完整敏感對話無限制送入模型。
- 製作清楚區分「PoC 已實作」與「Production Target」的 AWS 架構圖。
- 加入睡眠品質與環境溫度模型 / Tool，補齊題目背景中的裝置感測範圍。

P2：產品化

- 導入真實 OCR、身分驗證、同意管理、加密與資料保存政策。
- 串接正式通知服務及家屬端。
- 以實際使用數據驗證主動觸發率、回應率、Tool Calling 成功率與誤通知率。

11. 常見 Q&A

為什麼不用 LLM 直接判斷健康風險？

因為醫療風險需要可追溯、可測試、可調整的標準。LLM 適合做語言互動，不適合自由制定血壓、心率或漏服門檻。

藥盒開啟就代表有吃藥嗎？

不是。它只把可信程度從口頭回報提升為「感測訊號支持」，仍不能證明已吞服。

為什麼先做模擬 OCR？

MVP 要先驗證的不是 OCR 準確率，而是辨識錯誤能否被家屬校正、確認後能否安全進入提醒流程，以及是否具有完整稽核紀錄。

如果現場網路或 Gemini 失敗怎麼辦？

Memory backend 下的 OCR、事件、時間、服藥狀態與通知流程不依賴外部服務，可作為備援。正式上台前仍應補一條 scripted 對話或準備錄影，才能完整保留 Agent 故事。

13. 資料來源與盤點限制

- 題目依據：使用者提供的 Scenario、核心任務、三類詳細要求與 Deliverables 原文。
- 專案依據：README.md、PRD.md、SRS.md、docs/demo-runbook.md、原始碼、測試與 Git 歷史。
- docs/improvement-roadmap.md 的部分 PO 描述已落後於目前程式碼；本報告以實作與測試結果為準重新判定狀態。
- docs/TPM Assignment.pdf 已存在但尚未納入 Git；其 PDF 版面尚未在目前環境解析，但使用者已提供完整文字內容，本報告已依該文字逐項對齊。